

C++

Vojtěch Cimbura

Lecture II

Step by Step

04.11.2025

Quick Recap

- Last time:
 - Materials setup
 - **WIP** TetrisBlock Actor: C++ and BP
 - Setting material color, hiding / showing
- **TODO**
 - Finish TetrisBlock Actor C++
 - TetrisPlayer
 - Utilize Enhanced input
 - Create callbacks for input keybinds and verify functionality
 - TetrisGameMode
 - Board visualization
 - Delegate binding with Pawn
 - Implement game logic

TetrisBlock - Functions

- We'd like to have a basic cube mesh on the actor, assign the dynamic material to the mesh and be able to read and assign a mesh color
- **Constructor**
 - Load a default cube mesh from the engine
- **BeginPlay()**
 - Check if material is assigned
 - Create dynamic material instance and set the material to the mesh component
 - (Later) Set visibility of the actor to false
- Create Getter & Setter for **BlockColor** variable
 - Setter will also set material color - **SetVectorParameterValue()**
 - If the new color is **Black**, hide the actor
- Test the behavior by assigning a pseudorandom color to the block

TetrisBlock - BeginPlay (1/7)

- Don't forget to call the parent's class `BeginPlay()` function
 - When inheriting multiple classes, you will make sure that an initialization in parent classes is performed appropriately

```
// Called when the game starts or when spawned  
void ATetrisBlock::BeginPlay()  
{  
    Super::BeginPlay();  
}
```

UNREAL
ENGINE

TetrisBlock - BeginPlay (2/7)

- We should check if the material instance reference in the `BP_TetrisBlock` BP is assigned before we start using it in code
- Multiple options:
 - Runtime check of pointer validity together with Garbage Collection flag detection
 - In case of invalidity, we need to react to let the user know

```
if (IsValid(BlockMaterialInstance))
```

- Runtime check using Unreal's version of `assert()` function
 - In case of invalidity, an exception is thrown (see next slide for details and other options)

```
check(BlockMaterialInstance);
```

Asserts in UE

- `check()` / `checkf()`

- Behaves like `assert` in C/C++
- Failed condition **stops execution**, shows dialog, breakpoint
- Does not evaluate the condition in optimized build

- `verify()` / `verifyf()`

- Similar to `check`, **stops execution**
- Makes sure that evaluated condition is not optimized out in optimized build configurations

- `ensure()` / `ensureMsgf()`

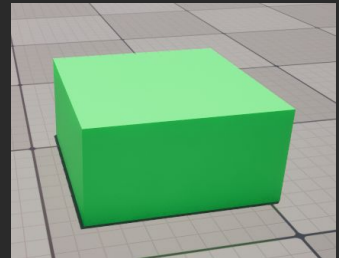
- Similar to `verify`, but **does not stop execution** -> non fatal errors
- Failure reported only once
 - Use `ensureAlways()` if you want to report always

TetrisBlock - BeginPlay (3/7)

- Dynamic Material Instance initialization
 - Creating the dynamic material instance out of the material instance assigned from `BP_TetrisBlock`
 - Setting the Cube's mesh material to be the newly created dynamic material instance (only one material, start on material slot index 0)

```
BlockMaterialInstanceDynamic = UMaterialInstanceDynamic::Create(BlockMaterialInstance, this);  
MeshComponent->SetMaterial(0, BlockMaterialInstanceDynamic);
```

- Now the `BP_TetrisBlock` will have the default color specified in the `MI_BlockMaterial` when dragged in into the scene in-game



TetrisBlock - BeginPlay (4/7)

- Testing and changing the color of the Dynamic Material Instance
 - First, create a Getter and Setter for the `BlockColor` variable in `ATetrisBlock`
 - Header file

```
void SetBlockColor(FColor InColor);  
const FColor GetBlockColor();
```

- Source file

```
void ATetrisBlock::SetBlockColor(FColor InColor)  
{  
    BlockColor = InColor;  
}  
  
const FColor ATetrisBlock::GetBlockColor()  
{  
    return BlockColor;  
}
```


TetrisBlock - BeginPlay (5/7)

- Then change the color in the Dynamic Material Instance in the Setter

```
void ATetrisBlock::SetBlockColor(FColor InColor)
{
    BlockColor = InColor;
    BlockMaterialInstanceDynamic->SetVectorParameterValue(BlockParameterName, BlockColor);
}
```

- This function called on the Dynamic Material Instance will change the value of named Vector Parameter to the specified color

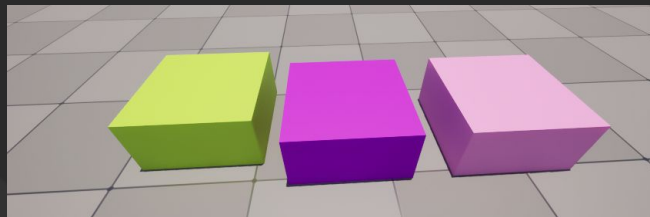
Global Vector Parameter		
BlockColor		
R		0,109897
G		1,0
B		0,175896
A		1,0

TetrisBlock - BeginPlay (6/7)

- Now in `ATetrisBlock::BeginPlay()`, after you initialize the Dynamic Material Instance, create a pseudo random `FColor` and set it as the new BlockColor
 - Create a random floating point number on interval $[0, 1)$ and map it to interval $[0-255]$ as unsigned 8-bit integer for each color's component (RGB)

```
uint8 R = static_cast<uint8>(FMath::SRand() * 255);  
uint8 G = static_cast<uint8>(FMath::SRand() * 255);  
uint8 B = static_cast<uint8>(FMath::SRand() * 255);  
  
SetBlockColor(FColor(R,G,B));
```

TetrisBlock - BeginPlay (7/7)



- After opening the editor, dragging a few `BP_TetrisBlock` actors into the scene and hitting Play, you should see each block having a different color
 - Completed `BeginPlay()` function

```
// Called when the game starts or when spawned
void ATetrisBlock::BeginPlay()
{
    Super::BeginPlay();

    check(BlockMaterialInstance);

    BlockMaterialInstanceDynamic = UMaterialInstanceDynamic::Create(BlockMaterialInstance, this);
    MeshComponent->SetMaterial(0, BlockMaterialInstanceDynamic);

    uint8 R = static_cast<uint8>(FMath::SRand() * 255);
    uint8 G = static_cast<uint8>(FMath::SRand() * 255);
    uint8 B = static_cast<uint8>(FMath::SRand() * 255);

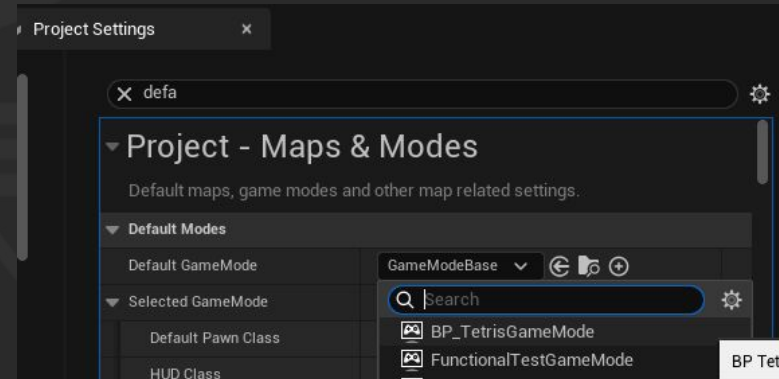
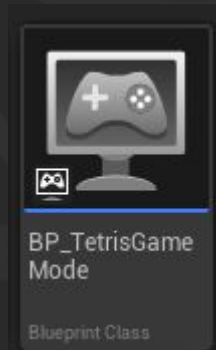
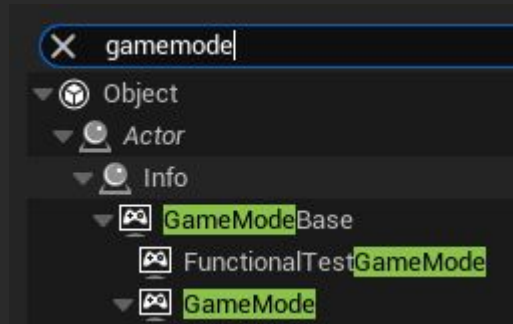
    SetBlockColor(FColor(R,G,B));
}
```

Next steps

- We have the board building block set up - player input and the game logic left
- Game Mode
 - Spawn the board
 - (Later) Put all of our game logic in there
 - (Later) Player's Pawn has access to the game mode:
 - Direct access - quick implementation, but messy solution
 - Delegates - slower implementation, but preferred way
- Player input
 - Utilize Enhanced Input
 - Setup input actions
 - Create `TetrisGameMode` and set up our custom Pawn
 - Print out debug messages into the Output Log to verify functionality

Editor - Create and Set GameMode

- Create new C++ class:
 - **TetrisGameMode** (GameModeBase)
 - Game mode is a level persistent class accessible from everywhere
 - (For level-transfer persistent class, use GameInstance)
- Create **BP_TetrisGameMode** based on the newly created C++ class
- Make the **BP_TetrisGameMode** the default game mode in Project Settings



TetrisGameMode - Overview

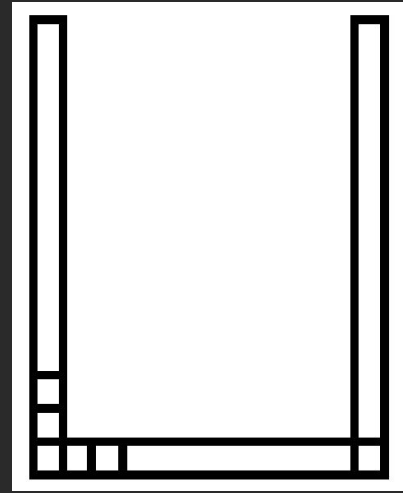
- Variables:

- `TSubclassOf<ATetrisBlock> BlockClassToSpawn;`
- `TArray<ATetrisBlock*> Board;`

- Functions:

- `BeginPlay()`

- Check if the variable `BlockClassToSpawn` is assigned
- Spawn **16x20** (WxH) grid of `BP_TetrisBlock` actors
 - Block size = 100 Unreal Units (distance between them)
 - Store spawned actors in the `Board` array
- After you verify blocks are spawned:
 - Set color of blocks to **White** unless the block is on a border, except the top row no-border blocks - then color is **Black**. When color is **Black**, hide the actor.



TetrisGameMode (1/8)

- Public variables in the `ATetrisGameMode` header file
 - `BlockClassToSpawn` is assignable in BP, do that afterwards in Editor
 - `TSubclassOf`: Type safe way of passing a reference to a Class, in our case, the `BP_TetrisBlock` Actor class from BP to Cpp
 - `Board` array will store spawned Actors

```
public:  
  
    UPROPERTY(EditAnywhere)  
    Changed in 0 Blueprints  
    TSubclassOf<ATetrisBlock> BlockClassToSpawn;  
  
    UPROPERTY()  
    Changed in 0 Blueprints  
    TArray<ATetrisBlock*> Board;
```

TetrisGameMode (2/8)

- Add function **BeginPlay()** in **ATetrisGameMode**
 - Header file - don't forget to override the base class's **BeginPlay()**

```
UCLASS()
0 derived Blueprint classes
class TETRIS_API ATetrisGameMode : public AGameModeBase
{
    GENERATED_BODY()

public:
    virtual void BeginPlay() override;
```

- RMB click on it and create a definition in the Source file
 - Don't forget to call the parent's **BeginPlay()**

```
void ATetrisGameMode::BeginPlay()
{
    Super::BeginPlay();
```


TetrisGameMode (3/8)

- Continue in `ATetrisGameMode::BeginPlay()`
 - Check validity of `BlockClassToSpawn` variable (if it is assigned in BP)
 - Define board dimension `WxH = 16x20` blocks
 - Define the block size: `100UU = 1m`, which is the default size of the Cube mesh we added to the `ATetrisBlock` Actor

```
check(BlockClassToSpawn);
```

```
const uint32 BoardWidth = 16u;  
const uint32 BoardHeight = 20u;  
const uint32 BlockSize = 100u;
```

TetrisGameMode (4/8)

- Continue in `ATetrisGameMode::BeginPlay()`
 - Iterate Bottom Left corner to Top Right corner by line and spawn `BP_TetrisBlock` actors using the passed in reference to the Class
 - As for Spawn Location, use X and Z axis and spawn in interval of the `BlockSize`, keep default actor rotation
 - Set the Block's color to **White** after spawning

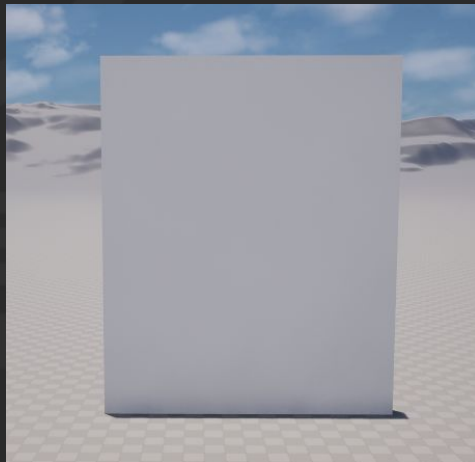
```
for (uint32 y = 0; y < BoardHeight; ++y)
{
    for (uint32 x = 0; x < BoardWidth; ++x)
    {
        const FVector Location(x * BlockSize, 0, y * BlockSize);
        ATetrisBlock* SpawnedBlock = GetWorld()->SpawnActor<ATetrisBlock>(
            BlockClassToSpawn, Location, FRotator::ZeroRotator);
        SpawnedBlock->SetBlockColor(FColor::White);
        Board.Emplace(SpawnedBlock);
    }
}
```

TetrisGameMode (5/8)

- Continue in `ATetrisGameMode::BeginPlay()`
 - Tip: You can preallocate the Board vector beforehand if you know how many elements will be there, to avoid vector reallocations

```
Board.Reserve(BoardWidth * BoardHeight);
```

- Hit Play and check if the board is spawned as intended:

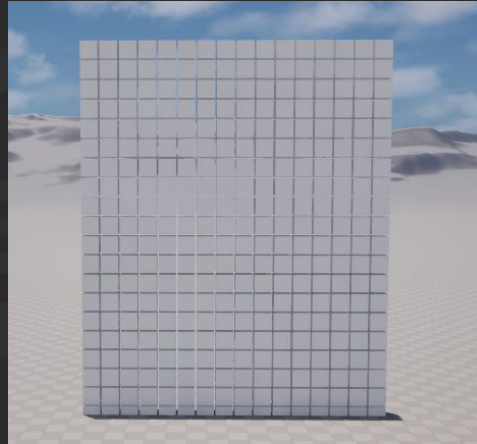


TetrisGameMode (6/8)

- We can see a White mass, individual blocks are not distinguishable. Fix that by scaling down the mesh by 10% in each axis
 - Head to `ATetrisBlock` constructor:

```
MeshComponent = CreateDefaultSubobject<UStaticMeshComponent>(TEXT("Cube"));  
MeshComponent->SetStaticMesh(CubeMesh);  
MeshComponent->SetWorldScale3D(FVector(0.9f));
```

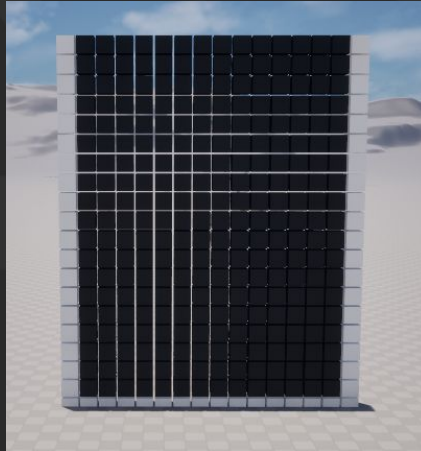
- Now check the result
 - Restart Editor if you see no change



TetrisGameMode (7/8)

- Lastly, define borders of the playing board
 - Set color of blocks to **White** unless the block is on a border, except the top row no-border blocks - then color is **Black**

```
const bool bBorder = x == 0 || x == BoardWidth - 1 || y == 0;  
SpawnedBlock->SetBlockColor(bBorder ? FColor::White : FColor::Black);
```

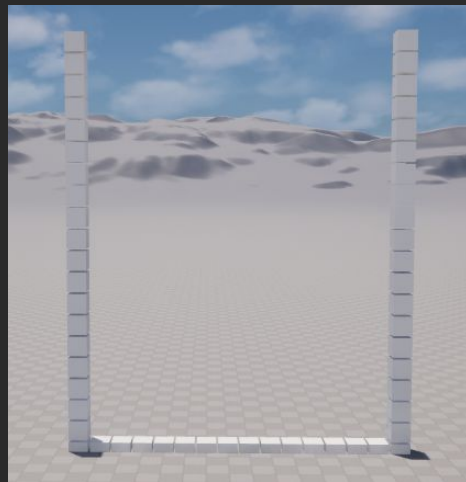


TetrisGameMode (8/8)

- Hide blocks that are **Black**

```
void ATetrisBlock::SetBlockColor(FColor InColor)
{
    BlockColor = InColor;

    if (BlockColor == FColor::Black)
    {
        SetActorHiddenInGame(true);
    }
    else
    {
        SetActorHiddenInGame(false);
        BlockMaterialInstanceDynamic->SetVectorParameterValue(BlockParameterName, BlockColor);
    }
}
```



Editor - Create and Set TetrisPawn

- Create new C++ class:
 - **TetrisPlayer** (Pawn)
- Create **BP_TetrisPlayer** based on the newly created C++ class
 - Set it as Default Pawn in the **BP_TetrisGameMode**
- When you hit Play, you won't be able to move around anymore
 - Move **PlayerStart** and rotate it in a way that will fit the board view



Pawn

A Pawn is an actor that can be 'possessed' and receive input from a controller.



BP_TetrisPlayer

Blueprint Class

Editor - Enhanced Input

- Create & Navigate to a folder 'Input' in the Content Browser
- Create mapping context: `IMC_TetrisMappingContext`
- Create 3 Input Actions:
 - `IA_MoveBlock`
 - Axis1D (float)
 - Triggers: Pressed
 - `IA_PlaceBlock`
 - Digital (bool)
 - Triggers: Pressed + Released
 - `IA_RotateBlock`
 - Digital (bool)
 - Triggers: Pressed
- Map them to keys of your choice in the `IMC_TetrisMappingContext`
 - A key: Negate modifier to move block left

Editor - Enhanced Input - IA_RotateBlock

 IA_Rotate x

Details x

Search

Description

Action Description

Action

Trigger when Paused

Reserve All Mappings

Value Type

Triggers

Index [0]

Actuation Threshold

Modifiers

Advanced

Accumulation Behavior

Input Consumption

Consume Lower Priority Enhanced Input Mappings

Consumes Action and Axis Mappings

Trigger Events That Consume Legacy Keys

User Settings

Player Mappable Key Settings

Digital (bool)

1 Array element

Pressed

0.5

0 Array element

Take Highest Absolute Value

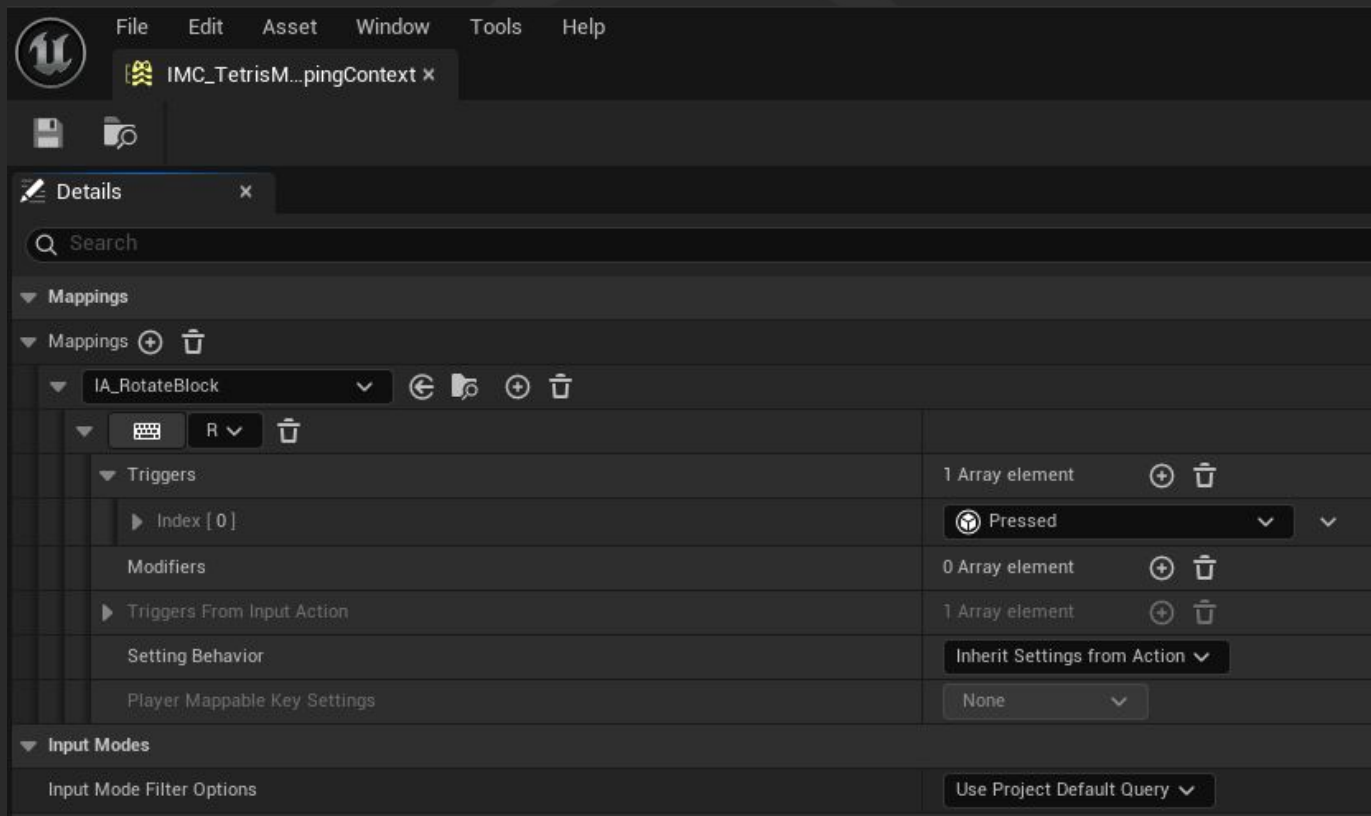
☒

☐

(No Flags Set)

None

Editor - Enhanced Input - IMC_TetrisMappingContext



Player's Pawn - Variables

- Connection to the Enhanced Input
 - Add includes in **TetrisPlayer.h**:

```
#include "EnhancedInputComponent.h"  
#include "EnhancedInputSubsystems.h"  
#include "InputActionValue.h"
```

- Add public member variables to **TetrisPlayer.h**
 - Similarly for other **UInputActions**

```
public:  
    UPROPERTY(EditAnywhere)  
    Changed in 0 Blueprints  
    UInputMappingContext* TetrisMappingContext;  
  
    UPROPERTY(EditAnywhere)  
    Changed in 0 Blueprints  
    UInputAction* RotateBlock;
```

Learning material

- [UE Naming convention](#)
- [Programming in UE C++, the whole documentation](#)
- [Balancing BPs and C++](#)
- [Getting started with Unreal Engine](#)
- [Asserts in UE](#)
- [Objects in UE](#)

UNREAL
ENGINE